

Sequential Logic



Building a Modern Computer From First Principles

1

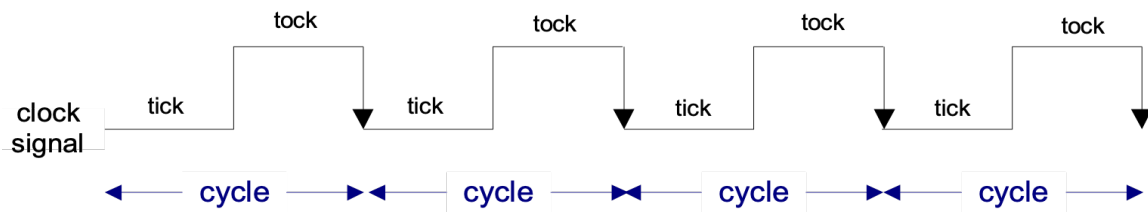
Sequential VS combinational logic

- **Combinational devices:** operate on data only; provide calculation services (e.g. Nand ... ALU)
- **Sequential devices:** operate on data and a clock signal; as such, can be made to be state-aware and provide storage and synchronization services
- Sequential devices are sometimes called "clocked devices"
- The low-level behavior of clocked / sequential gates is tricky
- The good news:
 - All sequential chips can be based on one low-level sequential gate, called "data flip flop", or DFF
 - The clock-dependency details can be encapsulated at the low-level DFF level
 - Higher-level sequential chips can be built on top of DFF gates using combinational logic only.

3

The Clock

HW
simulator
demo

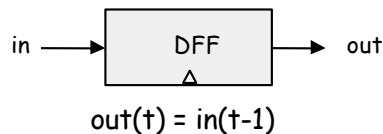


- In our jargon, a clock cycle = tick-phase (low), followed by a tock-phase (high)
- In real hardware, the clock is implemented by an oscillator
- In our hardware simulator, clock cycles can be simulated either
 - Manually, by the user, or
 - "Automatically," by a test script.

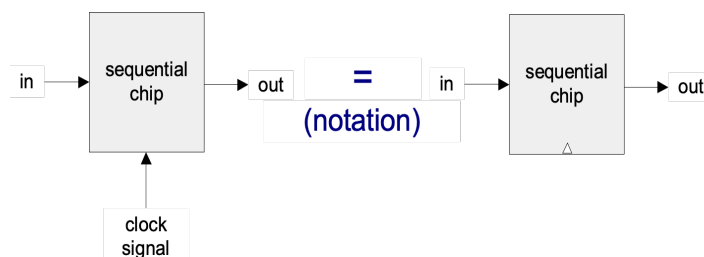
5

Flip-flop

HW
simulator
demo



- A fundamental state-keeping device
- For now, let us not worry about the DFF implementation
- Memory devices are made from numerous flip-flops, all regulated by the same master clock signal
- Notational convention:

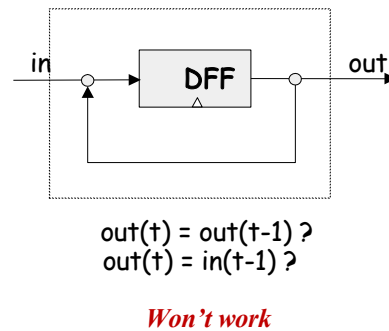
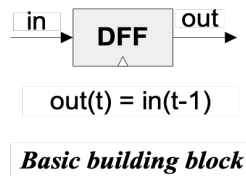
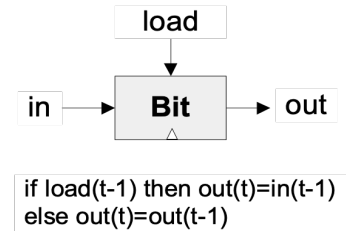


6

1-bit register (we call it "Bit")

Objective: build a storage unit that can:

- (a) Change its state to a given input
- (b) Maintain its state over time (until changed)

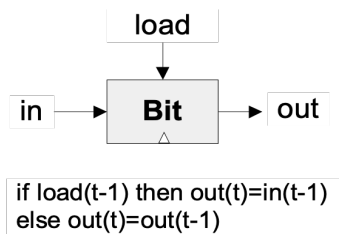


7

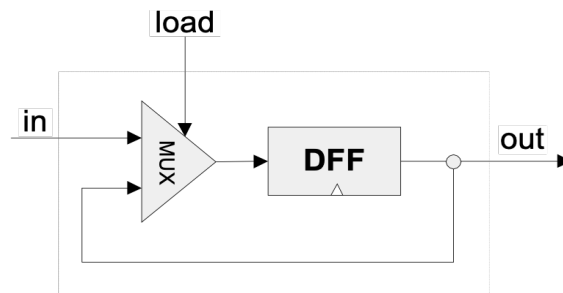
Bit register (cont.)

HW
simulator
demo

Interface



Implementation

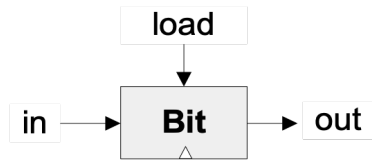


- Load bit
- Read logic
- Write logic

8

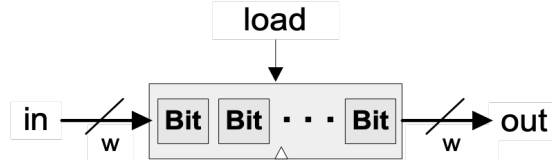
Multi-bit register

HW
simulator
demo



if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

1-bit register



if load(t-1) then out(t)=in(t-1)
else out(t)=out(t-1)

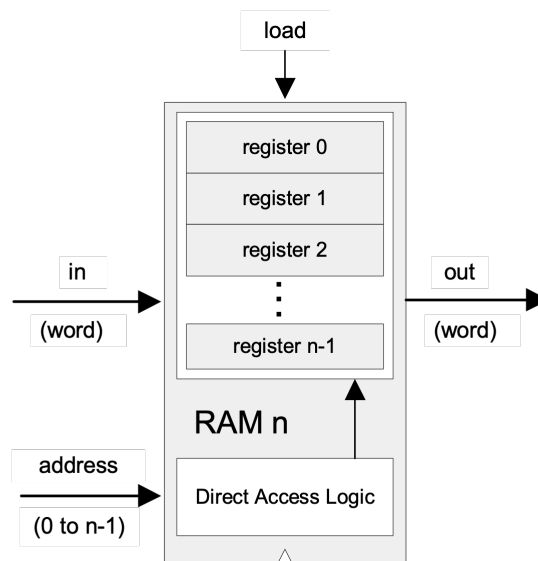
w-bit register

- Register's width: a trivial parameter
- Read logic
- Write logic

9

Random Access Memory (RAM)

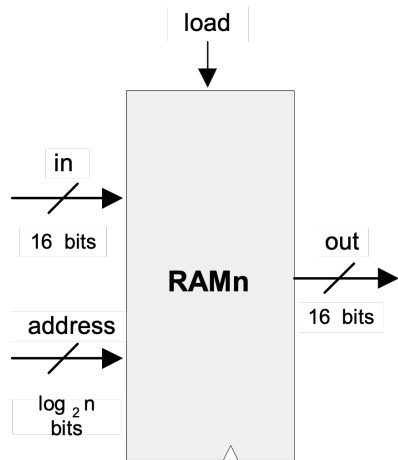
HW
simulator
demo



- Read logic
- Write logic.

11

RAM interface

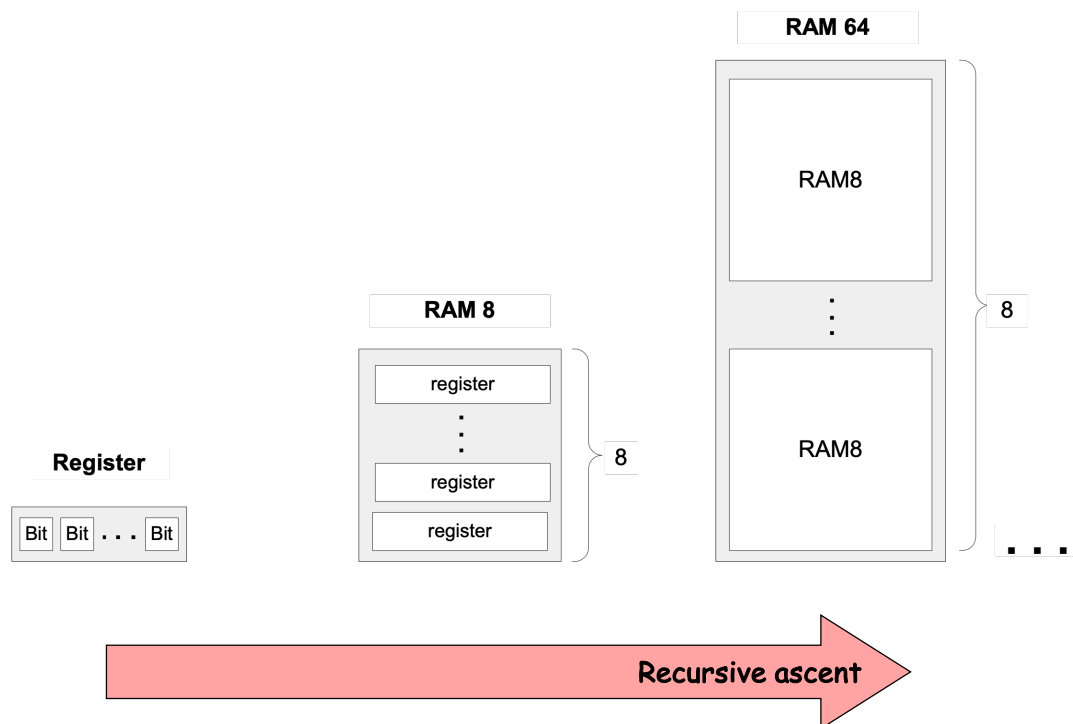


Chip name: RAMn // n and k are listed below
Inputs: in[16], address[k], load
Outputs: out[16]
Function: out(t)=RAM[address(t)](t)
 If load(t-1) then
 RAM[address(t-1)](t)=in(t-1)
Comment: "=" is a 16-bit operation.

The specific RAM chips needed for the Hack platform are:

Chip name	n	K
RAM8	8	3
RAM64	64	6
RAM512	512	9
RAM4K	4096	12
RAM16K	16384	14

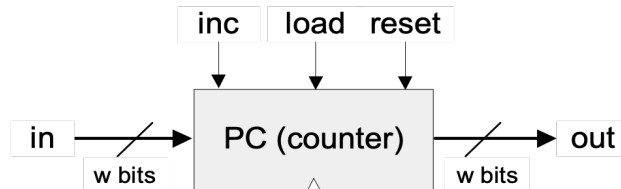
RAM anatomy



Counter

Needed: a storage device that can:

- (a) set its state to some base value
- (b) increment the state in every clock cycle
- (c) maintain its state (stop incrementing) over clock cycles
- (d) reset its state

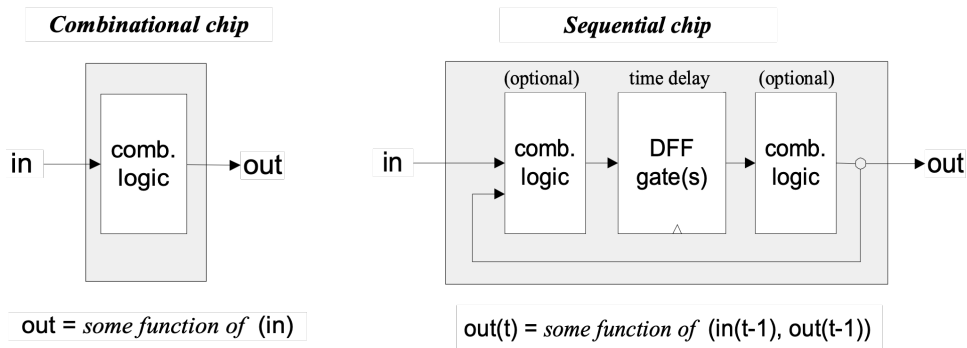


```
If reset(t-1) then out(t)=0
else if load(t-1) then out(t)=in(t-1)
else if inc(t-1) then out(t)=out(t-1)+1
else out(t)=out(t-1)
```

- Typical function: program counter
- Implementation: register chip + some combinational logic.

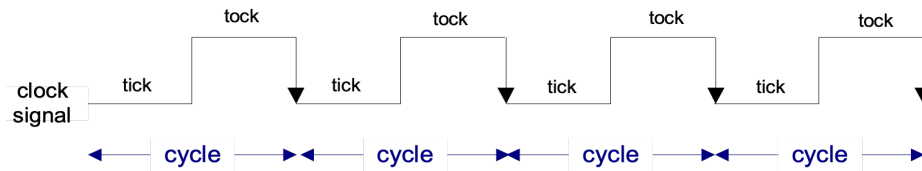
14

Recap: Sequential VS combinational logic

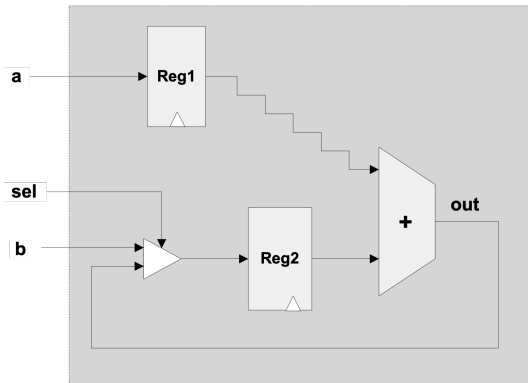


15

Time matters



- During a tick-tock cycle, the **internal states** of all the clocked chips are allowed to change, but their **outputs** are "latched"
- At the beginning of the next cycle, the **outputs** of all the clocked chips in the architecture **commit** to the new values.



Implications:

- ❑ Challenge: propagation delays
- ❑ Solution: clock synchronization
- ❑ Cycle length and processing speed.

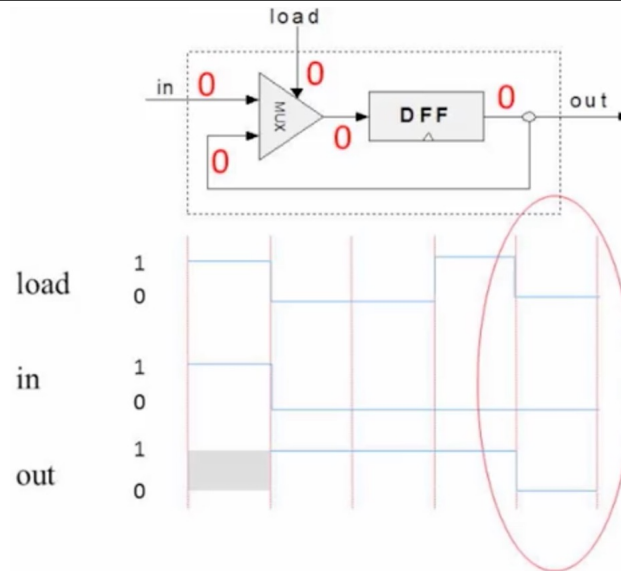
16

Project 3: sequential chips

Chip (HDL)	Description	Test script	Compare file
DFF	Data Flip-Flop (primitive)		
Bit	1-bit register	Bit.tst	Bit.cmp
Register	16-bit register	Register.tst	Register.cmp
RAM8	16-bit / 8-register memory	RAM8.tst	RAM8.cmp
RAM64	16-bit / 64-register memory	RAM64.tst	RAM64.cmp
RAM512	16-bit / 512-register memory	RAM512.tst	RAM512.cmp
RAM4K	16-bit / 4096-register memory	RAM4K.tst	RAM4K.cmp
RAM16K	16-bit / 16384-register memory	RAM16K.tst	RAM16K.cmp
PC	16-bit program counter	PC.tst	PC.cmp

17

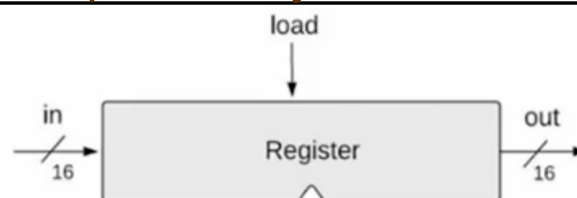
With a little help from my friends - Bit



- The DFF output must be reversed as a feedback
 - Note: in HDL a proper renaming must be done
"out" cannot be reversed as an input

18

With a little help from my friends - Register



```
/**  
 * 16-bit register:  
 * If load[t] == 1 then out[t+1] = in[t]  
 * else out does not change  
 */
```

```
CHIP Register {  
    IN in[16], load;  
    OUT out[16];  
    PARTS:  
        // Put your code here:
```

- Register can be realized by exploiting Bits
- To understand the clocked behavior, the built-in chip can be tested in the Hardware Simulator

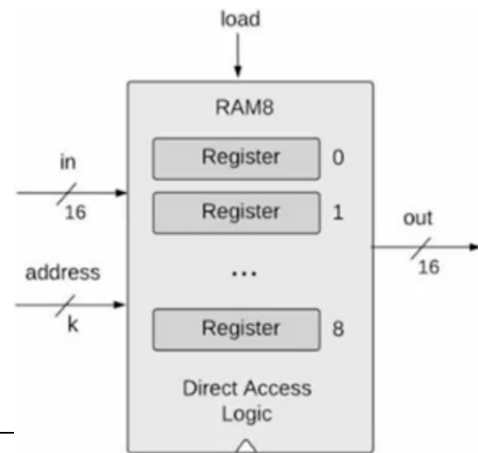
19

With a little help from my friends - RAM8

- Memory of 8 registers, each 16 bit-wide
- Implementation tips
 - Feed the "in" value to all the registers
 - Use Muxing/Demuxing logic to select the proper register (to be addressed and to be outputted)

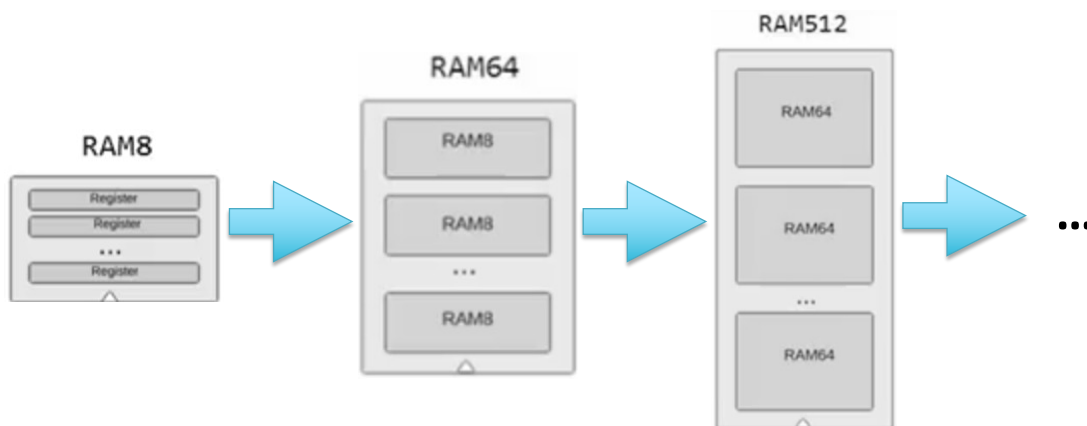
RAM8.hdl

```
CHIP RAM8 {  
    IN in[16], load, address[3];  
    OUT out[16];  
  
    PARTS:  
    // Put your code here:  
}
```



With a little help from my friends - Greater RAMs

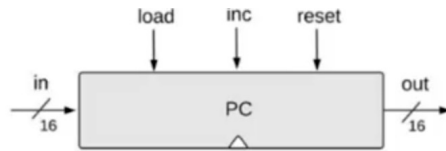
- Implementation tips:
 - the logic is the same as RAM8, but...
 - the "address" must be split into 2 fields
 - one to select a RAM chip
 - one to select a register within a RAM chip



With a little help from my friends - PC

■ Implementation tips:

- consider the priority of the 3 control bits
- use a register, an incrementer, some logic gates



PC.hdl

```
/**
 * A 16-bit counter with load, inc, and reset control bits.
 */
*
* if (reset[t]==1) out[t+1] = 0           // resetting: counter = 0
* else if (load[t]==1) out[t+1] = in[t]    // setting counter = value
*     else if (inc[t]==1) out[t+1] = out[t] + 1 // incrementing: counter++
*     else out[t+1] = out[t]              // counter does not change
*/

CHIP PC {
    IN in[16], load, inc, reset;
    OUT out[16];

    PARTS:
        // Implementation comes here.
}
```

PC = 0

PC = n

PC = PC+1

PC freezes

With a little help from my friends - delivery instructions

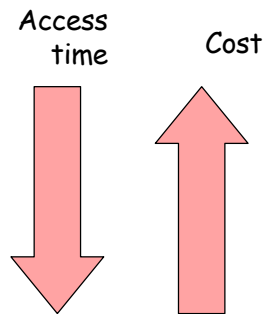
■ Please adhere to the following notation for delivering the .hdl files

- preserve the directory structure while working on your project files
 - Bit, Register, RAM8, RAM64, PC are inside "a"
 - RAM512, RAM4K, RAM16K are inside "b"
- submit two zip files
 - _P03CognomeNome_a.zip (containing immediately 5 hdl files, with no additional directory)
 - _P03CognomeNome_b.zip (containing immediately 3 hdl files, with no additional directory)

Perspective

- All the memory units described in this lecture are standard
- Typical memory hierarchy
 - SRAM ("static"), typically used for the cache
 - DRAM ("dynamic"), typically used for main memory
 - Disk

(Elaborate caching / paging algorithms)
- A Flip-flop can be built from Nand gates
- But ... real memory units are highly optimized, using a great variety of storage technologies.



End notes: some poetry about the limits of logic ...

There exists a field where things are neither true nor false;
I'll meet you there. (Rumi)

In the place where we are always right
No flowers will bloom in springtime (Yehuda Amichai)

A mind all logic is like a knife all blade;
It makes the hand bleed that uses it.
(Rabindranath Tagor)